DOCUMENTACIÓN TÉCNICA

Backend Django — Proyecto Vitrio

API REST para Sistema de Digital Signage

Samuel Ibáñez Rincón
Andrés Camilo Plazas
David Alejandro Cañon
Manuel Alejandro Betancurt
Diego Florez Rojas

Tabla de contenido

1. Arquitectura del Proyecto *4*
2. settings.py — Configuración Global *4*
2.1 INSTALLED_APPS 4
2.2 MIDDLEWARE 4
2.3 DATABASES 4
2.4 REST_FRAMEWORK 5
2.5 CORS 5
2.6 SIMPLE_JWT 5
2.7 CSRF_TRUSTED_ORIGINS 5
3. urls.py — Rutas de la API *5*
3.1 Router 5
3.2 Rutas de autenticación JWT 5
3.3 Rutas de documentación 5
4. models.py — Modelos de Base de Datos *6*
4.1 Cliente 6
4.2 Usuario 6
4.3 Credencial 6
4.4 Proyecto 6
4.5 Plantilla 7
4.6 Archivo 7
4.7 Dispositivo 7
5. serializers.py — Conversión de Datos *8*
6. views.py — Lógica del CRUD *8*
6.1 Estructura de cada ViewSet 8
6.2 Lo que genera ModelViewSet automáticamente 8
6.3 permission_classes = [IsAuthenticated] 8
7. Flujo completo de una petición *9*
8. Endpoints disponibles *9*
9. Variables de Entorno (.env) *10*
Librería utilizada: python-dotenv 10
Archivo .env — ubicado en la raíz del proyecto: 10

Cómo se carga en settings.py: 10

1. Arquitectura del Proyecto

El proyecto usa arquitectura de 3 capas con patrón MTV de Django:

Capa	Tecnología / Archivo
Presentación (Frontend)	React (desarrollado por otro equipo)
Lógica de negocio (API)	Django + Django REST Framework
Seguridad	JWT — SimpleJWT
Datos	MySQL
Documentación API	Swagger — drf-spectacular
Túnel desarrollo	ngrok

El frontend consume la API mediante peticiones HTTP con tokens JWT. Nunca accede directamente a la base de datos — todo pasa por el backend.

---

2. settings.py — Configuración Global

Archivo principal de configuración de Django. Contiene todas las variables que controlan el comportamiento del proyecto.

2.1 INSTALLED_APPS

Lista de aplicaciones activas en el proyecto:

```
'corsheaders'      # Permite peticiones desde el frontend (CORS)
'rest_framework'   # Django REST Framework para construir la API
'drf_spectacular'  # Genera documentación Swagger automáticamente
'VitrioApp'        # La app principal del proyecto
```

2.2 MIDDLEWARE

Lista de capas que procesan cada petición. El orden importa:

```
'corsheaders.middleware.CorsMiddleware'  # DEBE ir primero para CORS
```

2.3 DATABASES

Configuración de conexión a MySQL:

```
ENGINE   → 'django.db.backends.mysql'  # Motor de base de datos
NAME     → 'vitrio_db'                 # Nombre de la base de datos
USER     → 'root'                      # Usuario MySQL
PASSWORD → ''                          # Contraseña (vacía en desarrollo)
HOST     → 'localhost'                 # Servidor de base de datos
PORT     → '3306'                      # Puerto MySQL
```

2.4 REST_FRAMEWORK

Configuración de Django REST Framework:

```
DEFAULT_AUTHENTICATION_CLASSES → JWTAuthentication  # Autenticación por token JWT
DEFAULT_PERMISSION_CLASSES     → IsAuthenticated     # Todo requiere login
DEFAULT_SCHEMA_CLASS           → AutoSchema          # Para Swagger
```

2.5 CORS

Permite que el frontend (en otro dominio) consuma la API:

```
CORS_ALLOW_ALL_ORIGINS = True      # Permite cualquier origen en desarrollo
CORS_ALLOW_CREDENTIALS = True      # Permite enviar cookies y headers de auth
CORS_ALLOW_HEADERS = [...]         # Incluye header personalizado de ngrok
```

2.6 SIMPLE_JWT

Configuración de duración de los tokens de autenticación:

```
ACCESS_TOKEN_LIFETIME  = timedelta(hours=8)   # Token de acceso dura 8 horas
REFRESH_TOKEN_LIFETIME = timedelta(days=30)   # Token de refresco dura 30 días
```

2.7 CSRF_TRUSTED_ORIGINS

Dominios de confianza para peticiones externas (ngrok y localhost):

```
'https://presymphonic-nonnavigably-rhys.ngrok-free.dev'
'http://localhost:3000'   # Puerto React
'http://localhost:5173'   # Puerto Vite
```

---

3. urls.py — Rutas de la API

Define qué URL corresponde a qué vista. Usa un Router que genera automáticamente todas las rutas del CRUD.

3.1 Router

El DefaultRouter genera 5 rutas por cada modelo registrado:

```
GET    /api/clientes/        → listar todos
POST   /api/clientes/        → crear nuevo
GET    /api/clientes/{id}/   → ver uno
PUT    /api/clientes/{id}/   → actualizar
DELETE /api/clientes/{id}/   → eliminar
```

3.2 Rutas de autenticación JWT

```
POST /api/login/            → recibe usuario+contraseña, devuelve tokens
POST /api/token/refresh/    → recibe refresh token, devuelve nuevo access token
```

3.3 Rutas de documentación

```
GET /api/schema/   → esquema OpenAPI en JSON
```

```
GET /api/docs/     → interfaz Swagger interactiva
```

---

4. models.py — Modelos de Base de Datos

Cada clase representa una tabla en MySQL. Django crea y gestiona las tablas automáticamente mediante migraciones.

4.1 Cliente

Representa un negocio cliente del sistema (restaurante, tienda, etc.):

nombre	VARCHAR(200) — Nombre del negocio
email	VARCHAR(200) UNIQUE — Correo del negocio
telefono	VARCHAR(20) — Teléfono de contacto
fecha_creacion	TIMESTAMP — Se asigna automáticamente
fecha_actualizacion	TIMESTAMP — Se actualiza automáticamente

4.2 Usuario

Todos los usuarios del sistema con sus roles:

id_cliente	FK → Cliente (NULL si es admin interno)
fecha_nacimiento	DATE — Opcional
rol	ENUM: ADMINISTRADOR, ADMINISTRADOR_CLIENTE, CLIENTE

4.3 Credencial

Credenciales de acceso al sistema (relación 1 a 1 con Usuario):

id_usuario	OneToOne → Usuario
nombre	VARCHAR(200) — Nombre de usuario
contrasena	VARCHAR(255) — Contraseña encriptada
email	VARCHAR(200) UNIQUE — Email de acceso

4.4 Proyecto

Campaña o conjunto de contenido de un cliente:

id_cliente	FK → Cliente
nombre	VARCHAR(200)
descripcion	TEXT

fecha_inicio	DATE
fecha_entrega	DATE
estado	ENUM: PROPUESTO, ANALISIS, APROBADO, PLANIFICACION, EJECUCION, SEGUIMIENTO, PAUSADO, CANCELADO, COMPLETADO, CERRADO

4.5 Plantilla

Layouts prediseñados para mostrar en pantallas:

nombre	VARCHAR(200)
tipo	ENUM: VIDEO, IMAGEN, AUDIO, PAQUETE, METADATO, DOCUMENTO
version	VARCHAR(50) — Default: 1.0
estado	ENUM: ACTIVA, INACTIVA, DEPRECADA
ruta_base	VARCHAR(500) — Ruta del archivo base

4.6 Archivo

Videos, imágenes y documentos que se muestran en pantalla:

id_proyecto	FK → Proyecto
id_plantilla	FK → Plantilla (opcional)
nombre	VARCHAR(200)
ruta	VARCHAR(500) — Ruta del archivo
tipo	ENUM: VIDEO, IMAGEN, AUDIO, PAQUETE, METADATO, DOCUMENTO

4.7 Dispositivo

Pantallas físicas registradas en el sistema:

id_proyecto	FK → Proyecto
nombre	VARCHAR(200)
ubicacion	VARCHAR(200) — Opcional
token	VARCHAR(255) UNIQUE — Token de identificación
resolucion	VARCHAR(50) — Ej: 1920x1080
estado	ENUM: ACTIVO, INACTIVO, SIN_CONEXION
ultima_conexion	TIMESTAMP — Opcional

5. serializers.py — Conversión de Datos

Los serializers son el puente entre los modelos de Django y el formato JSON que usa la API. Se encargan de:

• Convertir objetos Python → JSON (para enviar al frontend)

• Convertir JSON → objetos Python (para guardar en la BD)

• Validar los datos recibidos antes de guardarlos

Cada modelo tiene su propio serializer con fields = '__all__' que incluye todos los campos automáticamente:

```
ClienteSerializer     → modelo Cliente
UsuarioSerializer     → modelo Usuario
CredencialSerializer  → modelo Credencial
ProyectoSerializer    → modelo Proyecto
PlantillaSerializer   → modelo Plantilla
ArchivoSerializer     → modelo Archivo
DispositivoSerializer → modelo Dispositivo
```

---

6. views.py — Lógica del CRUD

Cada ViewSet maneja automáticamente las 5 operaciones del CRUD para su modelo. Usando ModelViewSet se obtiene todo sin escribir cada método manualmente.

6.1 Estructura de cada ViewSet

```
class ClienteViewSet(viewsets.ModelViewSet):
    queryset           = Cliente.objects.all()    # Qué datos consultar
    serializer_class   = ClienteSerializer         # Cómo convertirlos a JSON
    permission_classes = [IsAuthenticated]         # Requiere token JWT
```

6.2 Lo que genera ModelViewSet automáticamente

list()	`GET /api/clientes/ — Lista todos los clientes`
create()	`POST /api/clientes/ — Crea un nuevo cliente`
retrieve()	`GET /api/clientes/{id}/ — Obtiene un cliente específico`
update()	`PUT /api/clientes/{id}/ — Actualiza un cliente`
destroy()	`DELETE /api/clientes/{id}/ — Elimina un cliente`

6.3 permission_classes = [IsAuthenticated]

Esta línea protege todos los endpoints — cualquier petición sin un token JWT válido recibirá un error 401 Unauthorized.

7. Flujo completo de una petición

Ejemplo: el frontend quiere crear un nuevo cliente.

Paso 1	Frontend hace POST /api/login/ con usuario y contraseña
Paso 2	Django verifica las credenciales y devuelve un access token JWT
Paso 3	Frontend guarda el token y hace POST /api/clientes/ con los datos + token en el header
Paso 4	urls.py recibe la petición y la dirige a ClienteViewSet
Paso 5	views.py verifica el token JWT (IsAuthenticated)
Paso 6	ClienteSerializer valida y convierte el JSON a objeto Python
Paso 7	models.py guarda el nuevo cliente en MySQL
Paso 8	ClienteSerializer convierte el objeto guardado a JSON
Paso 9	Frontend recibe el cliente creado con status 201

8. Endpoints disponibles

Método	Endpoint — Descripción
POST	/api/login/ — Obtener tokens JWT
POST	/api/token/refresh/ — Renovar access token
GET/POST	/api/clientes/ — Listar / Crear clientes
GET/PUT/DELETE	/api/clientes/{id}/ — Ver / Editar / Eliminar cliente
GET/POST	/api/usuarios/ — Listar / Crear usuarios
GET/PUT/DELETE	/api/usuarios/{id}/ — Ver / Editar / Eliminar usuario
GET/POST	/api/proyectos/ — Listar / Crear proyectos
GET/PUT/DELETE	/api/proyectos/{id}/ — Ver / Editar / Eliminar proyecto

GET/POST	`/api/plantillas/ — Listar / Crear plantillas`
GET/PUT/DELETE	`/api/plantillas/{id}/ — Ver / Editar / Eliminar plantilla`
GET/POST	`/api/archivos/ — Listar / Crear archivos`
GET/PUT/DELETE	`/api/archivos/{id}/ — Ver / Editar / Eliminar archivo`
GET/POST	`/api/dispositivos/ — Listar / Crear dispositivos`
GET/PUT/DELETE	`/api/dispositivos/{id}/ — Ver / Editar / Eliminar dispositivo`
GET	`/api/docs/ — Documentación Swagger interactiva`

9. Variables de Entorno (.env)

Las variables de entorno permiten separar la información sensible del código fuente, evitando que datos como contraseñas o claves secretas queden expuestos en el repositorio de GitHub.

Librería utilizada: python-dotenv

Archivo .env — ubicado en la raíz del proyecto:

```
SECRET_KEY=django-insecure-tu-clave-aqui
DB_NAME=vitrio_db
DB_USER=root
DB_PASSWORD=
DB_HOST=localhost
DB_PORT=3306
```

Cómo se carga en settings.py:

```
pythonfrom dotenv import load_dotenv
import os
load_dotenv()  # Carga el archivo .env

SECRET_KEY = os.getenv('SECRET_KEY')

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME':     os.getenv('DB_NAME'),
        'USER':     os.getenv('DB_USER'),
        'PASSWORD': os.getenv('DB_PASSWORD'),
        'HOST':     os.getenv('DB_HOST'),
        'PORT':     os.getenv('DB_PORT'),
  }
}
```

Nota ¡!: Cuando otro desarrollador clone el proyecto debe crear su propio `.env` con sus datos locales para que funcione correctamente.